

Фамилия: Филибер

Имя: Никола

Группа: НФИмд-02-25

Адрес электронной почты: 1132259430@rudn.ru

ЛАБОРАТОРНАЯ РАБОТА N°2

Шифры перестановки

1. Маршрутное шифрование.

нелзя недооценивать противника —————> ЕЕНПНЗОАТАЬОВОКИНЕЬВЯЦТИА.

n = 6 m = 5 X- 29 букв

строки и столбцы

нелъ3Я
недоои
ениат
бпроти
вникаХ
пароль
4 1 6 3 2 5

```
# Python3 шифрование (Route Cipher)
import math

ALPHABET = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"

def normalize(text: str) -> str:
    text = text.upper()
    return "".join(ch for ch in text if ch in ALPHABET)

def column_order(key: str):
    """
    Returns column indices in the order they should be read,
    based on alphabetical sorting of key letters.
    """
    key = key.upper()
    indexed = list(enumerate(key)) # [(0, 'Z'), (1, 'E'), ...]
    indexed_sorted = sorted(indexed, key=lambda x: (x[1], x[0]))
    return [idx for idx, _ in indexed_sorted]

def columnar_encrypt(plaintext: str, key: str, pad: str = "X") -> str:
    pt = normalize(plaintext)
    key = normalize(key)
    if not key:
        raise ValueError("Key must not be empty")

    cols = len(key)
    rows = math.ceil(len(pt) / cols)
    pt = pt.ljust(rows * cols, pad)

    # Fill row by row
    grid = [list(pt[r*cols:(r+1)*cols]) for r in range(rows)]

    # Read columns in key-sorted order
    order = column_order(key)
    out = []
    for c in order:
        for r in range(rows):
            out.append(grid[r][c])
    return "".join(out)

def columnar_decrypt(ciphertext: str, key: str) -> str:
    ct = normalize(ciphertext)
    key = normalize(key)
    if not key:
        raise ValueError("Key must not be empty")

    cols = len(key)
    if len(ct) % cols != 0:
        raise ValueError("Ciphertext length must be a multiple of key length")

    rows = len(ct) // cols
    order = column_order(key)

    # Create empty grid
    grid = [[""] * cols for _ in range(rows)]

    # Fill columns in the same order they were read
    pos = 0
    for c in order:
        for r in range(rows):
            grid[r][c] = ct[pos]
            pos += 1

    # Read row by row
    return "".join("".join(row) for row in grid)

# --- demo ---
text = "WE MUST NOT UNDERESTIMATE THE ENEMY"
key = "HELLO"
c = columnar_encrypt(text, key)
p = columnar_decrypt(c, key)
print("COLUMNAR")
print("Plain :", text)
print("Key   :", key)
print("Cipher:", c)
print("Back  :", p)
```

COLUMNAR
Plain : WE MUST NOT UNDERESTIMATE THE ENEMY
Key : HELLO
Cipher: ENDTENWTNSTEMOEITEUTRMHMSUEAEY
Back : WEMUSTNOTUNDERESTIMATETHEENEMY